



Universidade Federal De Mato Grosso Do Sul
Campus de Três Lagoas
Bacharelado em Sistemas de Informação
Banco de Dados

Relatório Técnico

Tema: Sistemas para Gerenciamento de Torneios de Futebol.

Título: SoccerData - Projeto de Banco de Dados para Torneios de Futebol.

Professora: Ivone Penque Matsuno Yugoshi

Gabriel Fiorentino Bressane

Três Lagoas - MS

2025

Sumário

1. Introdução.....	4
1.1. Estudo de Caso.....	4
2. Esquema Conceitual.....	6
2.1. Diagrama Entidade-Relacionamento.....	6
2.2. Diagrama Relacional.....	7
3. Esquema Lógico.....	7
3.1. Domínio.....	7
Tabela 1 - Pessoa.....	8
Tabela 2 - Telefone_Pessoa.....	8
Tabela 3- Email_Pessoa.....	9
Tabela 4 - Arbitro.....	9
Tabela 5 - Jogador.....	10
Tabela 6 - Treinador.....	11
Tabela 7 - Tabela Time.....	11
Tabela 8 - Comissao_Tecnica.....	12
Tabela 9 - Elenco.....	13
Tabela 10 - Torneio.....	13
Tabela 11 - Estadio.....	14
Tabela 12 - Arbitragem.....	15
Tabela 13 - Partida.....	15
Tabela 14 - EventoPartida.....	17
Tabela 15 - Classificação.....	18
Tabela 16 - Escalacao_time.....	19
4. Script Criação do Banco de Dados.....	19
4.1. Script Criação Tabelas.....	19
Código-Fonte 1 - Pessoa.....	19
Código-Fonte 2 - Tabela Telefone_Pessoa.....	20
Código-Fonte 3 - Tabela Email_Pessoa.....	20

Código-Fonte 4 - Tabela Arbitro.....	20
Código-Fonte 5 - Tabela Jogador.....	20
Código-Fonte 6 - Tabela Treinador.....	21
Código-Fonte 7 - Tabela Time.....	21
Código-Fonte 8 - Tabela ComissaoTecnica.....	22
Código-Fonte 9 - Tabela Elenco.....	22
Código-Fonte 10 - Tabela Torneio.....	22
Código-Fonte 11 - Tabela Estadio.....	23
Código-Fonte 12 - Tabela Arbitragem.....	23
Código-Fonte 13 - Tabela Partida.....	24
Código-Fonte 14 - Tabela EventoPartida.....	24
Código-Fonte 15 - Tabela Classificacao.....	25
Código-Fonte 16 - Tabela Escalacao_time.....	25
5. Scripts para Consulta.....	25
5.1. Consulta 1 - Ver todas as partidas com detalhes completos.....	25
5.2. Consulta 2 - Gols marcados como mandante no Brasileiro Feminino A1.....	26
5.3. Consulta 3 - Jogos dos quatro grandes de SP já encerrados.....	27
5.4. Consulta 4 - Árbitros que mais apitaram partidas no Campeonato Brasileiro Série A Masculino 2025.....	27
5.5. Consulta 5 - Estádios que nunca sediaram uma partida.....	28
5.6. Consulta 6 - Artilharia do Brasileirão Série A Masculino 2025.....	28
5.7. Consulta 7 - Estádios com maior público médio.....	29
5.8. Consulta 8 - Jogadores com mais tempo de clube.....	29
5.9. Consulta 9 - Classificação atual do Brasileiro Feminino A1 2025.....	30
5.10. Consulta 10 - Times com menos cartões recebidos.....	31
6. Gatilhos.....	31
6.1. Gatilho 1 - AtualizarClassificacao.....	31
6.2. Gatilho 2 - ImpedirDuploVinculo.....	33
6.3. Gatilho 1 - ValidarEventoPartida.....	34
7. Procedimentos Armazenados (Stored Procedure).....	35
7.1. Procedure 1 - CadastrarJogador.....	35

7.2. Procedure 2 - TransferirJogador.....	37
7.3. Procedure 3 - FinalizarPartida.....	38
8. Funções.....	38
8.1. Função 1 - Gols time torneio.....	38
8.2. Função 2 - Partidas vencidas pelo time em um torneio.....	40
8.3. Função 3 - Top 10 jogadores com mais gol pelo clube.....	41

1. Introdução

O futebol é um dos esportes mais populares do mundo, movimentando milhões de pessoas e envolvendo toda uma estrutura de organização que vai desde jogadores e treinadores até árbitros, estádios e torneios. Com o avanço da tecnologia e a crescente necessidade de precisão nas informações, torna-se cada vez mais importante o uso de sistemas que auxiliem na gestão e controle de dados esportivos.

Neste contexto, surge a ideia do **SoccerData**, como uma proposta de sistema de banco de dados voltado para o gerenciamento de torneios de futebol, visando otimizar o registro, a consulta e a administração das informações relacionadas às competições. O sistema busca eliminar falhas comuns em processos manuais, como perda de dados e inconsistências de registro, garantindo maior confiabilidade, agilidade e transparência na gestão esportiva.

Assim, o desenvolvimento do *SoccerData* representa uma aplicação prática da tecnologia da informação na área esportiva, demonstrando como a organização estruturada de dados pode contribuir significativamente para a eficiência administrativa e para o aprimoramento da experiência de gestão de torneios.

1.1. Estudo de Caso

O **SoccerData** é um sistema voltado para o gerenciamento de torneios de futebol, desenvolvido com o objetivo de facilitar a gestão, a consulta e o registro das informações de cada campeonato. A administração de campeonatos é, muitas vezes, realizada de forma manual, o que pode resultar em perda de dados, marcações incorretas e até mesmo em campeões definidos de maneira equivocada, além de, em alguns casos, não registrar corretamente jogadores ou equipes.

O sistema visa resolver essas inconsistências de dados relacionadas a jogadores, campeonatos, partidas, arbitragens, treinadores, times e estádios. Para isso, diversas informações são coletadas e estruturadas de forma organizada dentro do banco de dados.

No cadastro de jogadores, são armazenadas informações pessoais como CPF, primeiro nome, nome do meio (opcional), último nome, data de nascimento, nacionalidade, sexo, telefone e e-mail. Além disso, o sistema registra dados específicos do atleta, como posição em campo, altura, peso e perna de preferência, armazenando os detalhes de cada jogador.

Para os treinadores, também são coletadas as informações básicas de identificação, juntamente com dados profissionais, como o tipo de licença e o número da licença de treinador, assegurando o controle de profissionais habilitados para comandar equipes.

No caso dos árbitros, o sistema armazena informações que permitem identificar e acompanhar sua atuação nas competições. Além dos dados pessoais, são registradas informações profissionais, como a federação à qual o árbitro está vinculado, o tipo de licença que possui e o status de atividade. Cada árbitro pode atuar em diferentes funções dentro de uma partida, como árbitro principal, assistente ou quarto árbitro, não sendo exclusivo a apenas uma dessas posições.

O cadastro de times armazena as informações essenciais de cada equipe participante dos torneios. São registrados o nome, sigla, cidade de origem e a data de fundação, permitindo uma identificação completa e padronizada dos clubes.

O elenco representa a relação entre os jogadores e os times aos quais pertencem em um determinado período. Cada registro contém o jogador vinculado, o time, o número da camisa, a data de início e a data de término da participação. Essa tabela serve para manter o histórico das formações de cada time e garantir que apenas jogadores devidamente registrados possam atuar em partidas oficiais.

O torneio é a entidade que centraliza as informações sobre cada campeonato administrado pelo sistema. Nela são definidos o nome, tipo, o ano de realização, o país e o status. Essa tabela permite o acompanhamento simultâneo de diferentes competições. A tabela do estádio registra os locais onde as partidas são realizadas. São armazenados dados como nome, cidade, capacidade e o dono da instalação.

A arbitragem é composta pela equipe de árbitros responsável pela condução das partidas. O sistema registra o árbitro principal, dois assistentes e o quarto árbitro, todos referenciados na tabela de árbitros. Essa modelagem assegura o controle sobre quais profissionais atuaram em cada partida e mantém o histórico completo de suas designações.

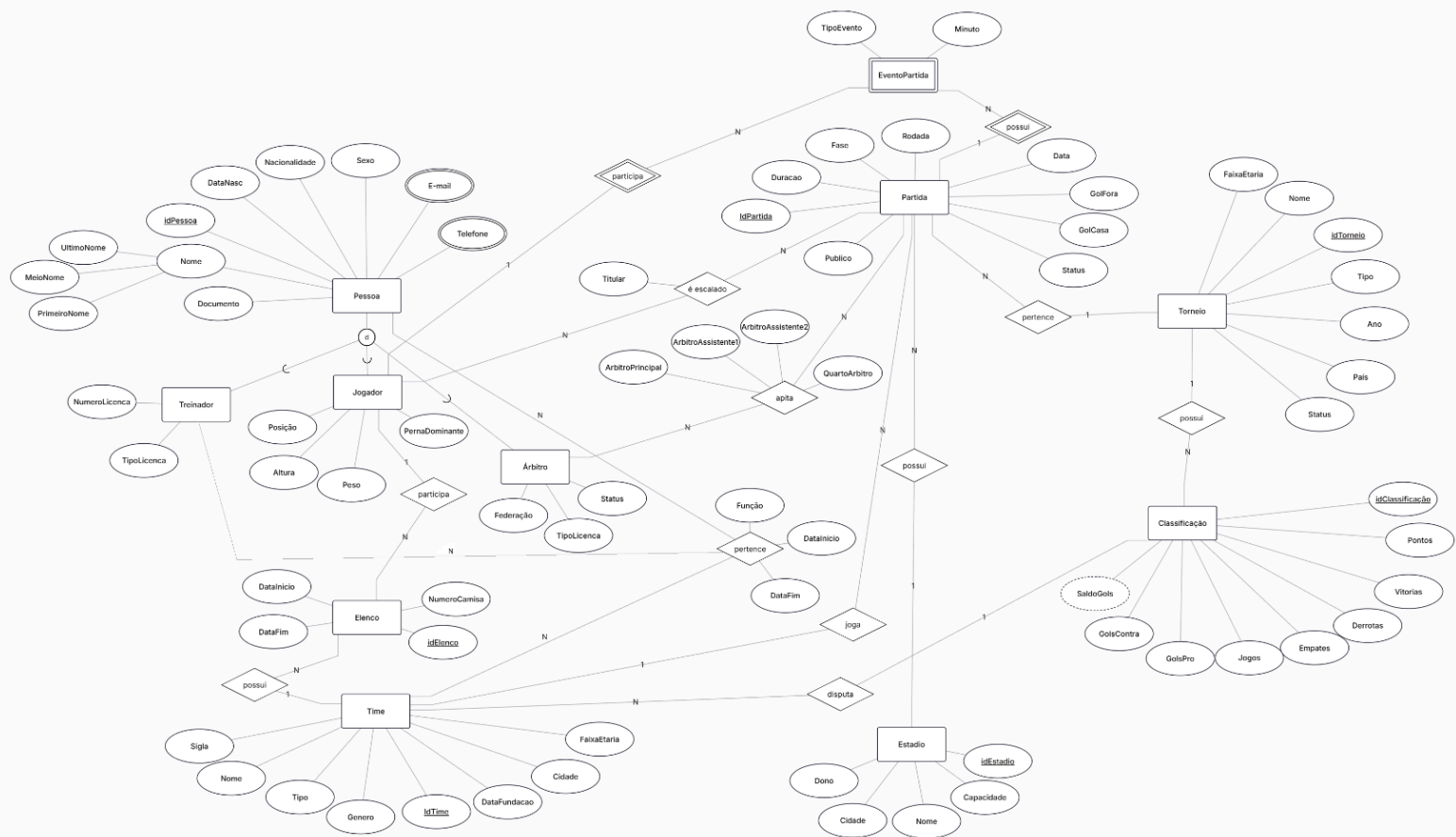
As partidas reúnem as informações sobre os confrontos realizados dentro dos torneios. Cada registro inclui os times envolvidos, o estádio, a arbitragem, a data e hora da partida, a fase do torneio, o placar final, o status e o público presente. Essa estrutura é a base para gerar estatísticas e resultados oficiais de cada competição. Os eventos da partida detalham as ocorrências que acontecem durante um jogo,

como gols, cartões, substituições e pênaltis. Cada evento é registrado com o minuto, o jogador envolvido, o time, o tipo de evento e a ordem em que ocorreu.

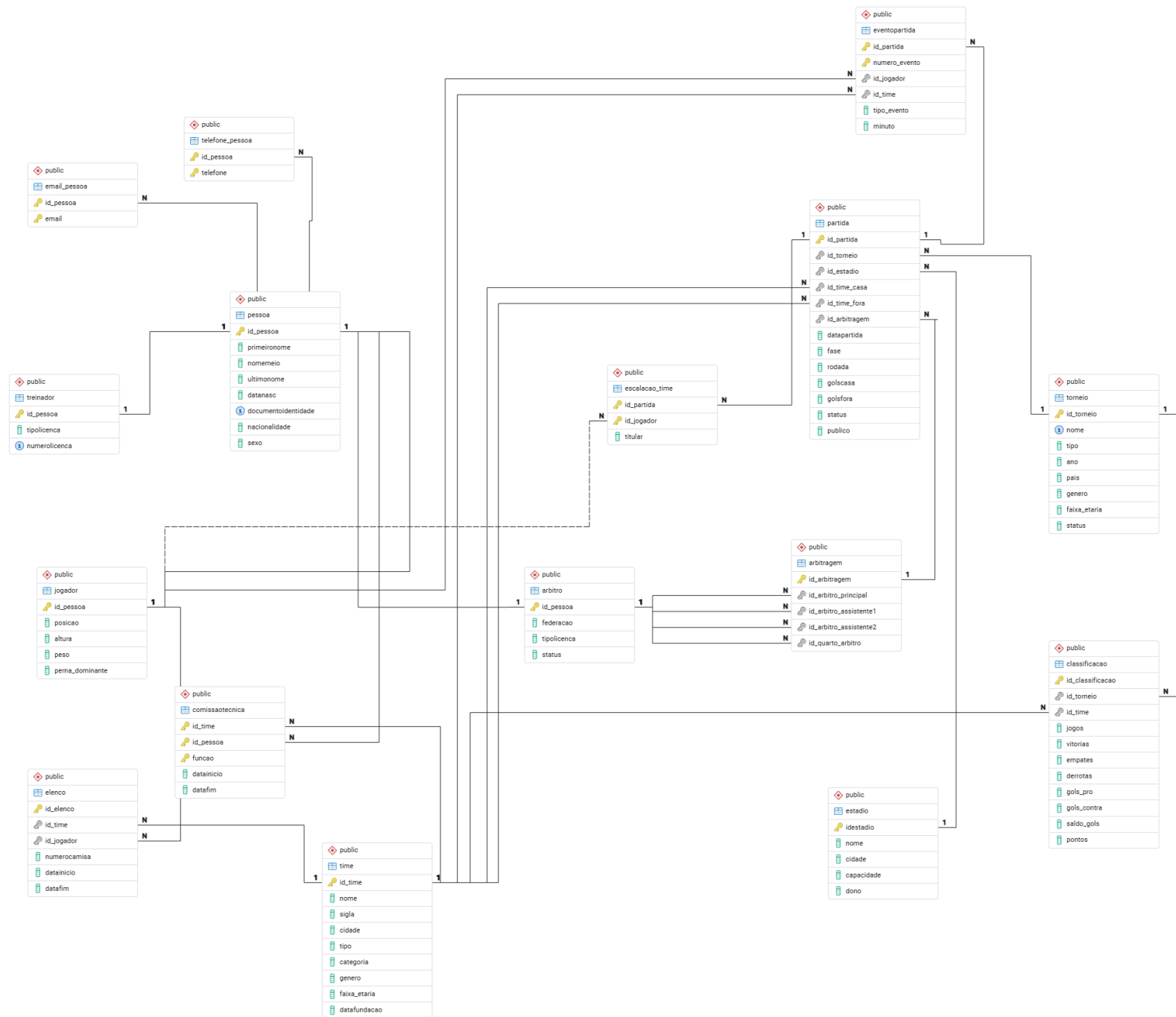
Por fim, a tabela de classificação mantém o desempenho de cada time dentro de um torneio. São armazenados indicadores como jogos disputados, vitórias, empates, derrotas, gols pró, gols contra, saldo de gols e pontos. Com base nesses dados, o sistema é capaz de gerar automaticamente a tabela de classificação, facilitando o acompanhamento da competição e a definição dos líderes

2. Esquema Conceitual

2.1. Diagrama Entidade-Relacionamento



2.2. Diagrama Relacional



3. Esquema Lógico

3.1. Domínio

Tabela 1 - Pessoa

Atributo	Tipo de Dados	Restrição de Domínio	Restrição de Entidade	Restrição Referencial
id_pessoa	INTEGER	Obrigatório	Chave Primária	-
primeiroNome	VARCHAR(60)	Obrigatório	-	-
nomeMeio	VARCHAR(50)	Opcional	-	-
ultimoNome	VARCHAR(60)	Obrigatório	-	-
dataNasc	DATE	Obrigatório	-	-
documento de entidade	VARCHAR(20)	Obrigatório	Único	-
nacionalidade	VARCHAR(50)	Obrigatório	-	-
sexo	CHAR(1)	Obrigatório e tem que ser 'M' ou 'F'	-	-

Tabela 2 - Telefone_Pessoa

Atributo	Tipo de Dados	Restrição de Domínio	Restrição de Entidade	Restrição Referencial
id_pessoa	INTEGER	Obrigatório	Chave Primária	Sim, da tabela Pessoa
telefone	VARCHAR(20)	Obrigatório	Chave Primária	-

Tabela 3- Email_Pessoa

Atributo	Tipo de Dados	Restrição de Domínio	Restrição de Entidade	Restrição Referencial
id_pessoa	INTEGER	Obrigatório	Chave Primária	Sim, da tabela Pessoa
email	VARCHAR(60)	Obrigatório	Chave Primária	-

Tabela 4 - Arbitro

Atributo	Tipo de Dados	Restrição de Domínio	Restrição de Entidade	Restrição Referencial
id_pessoa	INTEGER	Obrigatório	Chave Primária	Sim, da tabela Pessoa
federacao	VARCHAR(60)	Obrigatório	-	-
tipoLicenca	VARCHAR(60)	Obrigatório e tem que ser ('Ativo','Suspendido','Aposentado'), valor padrão é 'Ativo'	-	-

Tabela 5 - Jogador

Atributo	Tipo de Dados	Restrição de Domínio	Restrição de Entidade	Restrição Referencial
id_pessoa	INTEGER	Obrigatório	Chave Primária	Sim, da tabela Pessoa
posicao	VARCHAR(50)	Obrigatório e que ser: ('Goleiro', 'Zagueiro', 'Lateral Direito', 'Lateral Esquerdo', 'Volante', 'Meia Atacante', 'Meia Central', 'Ponta Direita', 'Ponta Esquerda', 'Centroavante', 'Segundo Atacante')	-	-
altura	DECIMAL(4,2)	Obrigatório	-	-
peso	DECIMAL(4,2)	Obrigatório	-	-
perna_dominante	VARCHAR(50)	Obrigatório e que ser: ('Direita',	-	-

		'Esquerda' ou 'Ambas')		
--	--	---------------------------	--	--

Tabela 6 - Treinador

Atributo	Tipo de Dados	Restrição de Domínio	Restrição de Entidade	Restrição Referencial
id_pessoa	INTEGER	Obrigatório	Chave Primária	Sim, da tabela Pessoa
tipoLicenca	VARCHAR(60)	Obrigatório	-	-
numeroLicenca	INTEGER	Obrigatório	Único	-

Tabela 7 - Tabela Time

Atributo	Tipo de Dados	Restrição de Domínio	Restrição de Entidade	Restrição Referencial
id_time	INTEGER	Obrigatório	Chave Primária	-
nome	VARCHAR(100)	Obrigatório	-	-
sigla	VARCHAR(3)	Obrigatório	-	-
cidade	VARCHAR(100)	Obrigatório	-	-
tipo	VARCHAR(50)	Obrigatório e tem que ser	-	-

		('Clube', 'Seleção Nacional')		
categoria	VARCHAR(50)	Obrigatório e tem que ser ('Profissional', 'Amador', 'Base')	-	-
faixa_etaria	VARCHAR(50)	Opcional e tem que ser ('Principal', 'Sub-20', 'Sub-17', 'Sub-15')	-	-
dataFundaca o	DATE	Obrigatório	-	-

Tabela 8 - Comissao_Tecnica

Atributo	Tipo de Dados	Restrição de Domínio	Restrição de Entidade	Restrição Referencial
id_time	INTEGER	Obrigatório	Chave Primária	Sim, vem da tabela Time
id_pessoa	INTEGER	Obrigatório	Chave Primária	Sim, vem da tabela Pessoa
funcao	VARCHAR(50)	Obrigatório e tem que ser ('Treinador', 'Auxiliar Técnico',	-	-

		'Massagista', 'Preparador Físico')		
dataInicio	DATE	Obrigatório	-	-
dataFim	DATE	Opcional	-	-

Tabela 9 - Elenco

Atributo	Tipo de Dados	Restrição de Domínio	Restrição de Entidade	Restrição Referencial
id_elenco	INTEGER	Obrigatório	Chave Primária	-
id_time	INTEGER	Obrigatório	Único	Sim, vem da tabela Time
id_jogador	INTEGER	Obrigatório	Único	Sim, vem da tabela Jogador
numeroCamisa	INTEGER	Obrigatório	-	-
dataInicio	DATE	Obrigatório	-	-
dataFim	DATE	Opcional	-	-

Tabela 10 - Torneio

Atributo	Tipo de Dados	Restrição de Domínio	Restrição de Entidade	Restrição Referencial
id_torneio	INTEGER	Obrigatório	Chave Primária	-
nome	VARCHAR (100)	Obrigatório	Único	-

tipo	VARCHAR (50)	Obrigatório, valores: ('Pontos Corridos', 'Mata-Mata', 'Grupos', 'Amistoso')	-	-
ano	INTEGER	Obrigatório	-	-
pais	VARCHAR (50)	Obrigatório	-	-
genero	VARCHAR (50)	Obrigatório e tem que ser ('Masculino', 'Feminino')	-	-
faixa_etaria	VARCHAR (50)	Opcional e tem que ser ('Principal', 'Sub-20', 'Sub-17', 'Sub-15')	-	-

Tabela 11 - Estadio

Atributo	Tipo de Dados	Restrição de Domínio	Restrição de Entidade	Restrição Referencial
idEstadio	INTEGER	Obrigatório	Chave Primária	-
nome	VARCHAR(100)	Obrigatório	-	-
cidade	VARCHAR(100)	Obrigatório	-	-
capacidade	INTEGER	Obrigatório	-	-

dono	VARCHAR(100)	Obrigatório	-	-
------	--------------	-------------	---	---

Tabela 12 - Arbitragem

Atributo	Tipo de Dados	Restrição de Domínio	Restrição de Entidade	Restrição Referencial
id_arbitragem	INTEGER	Obrigatório	Chave Primária	-
id_arbitro_principal	INTEGER	Obrigatório	-	Sim, vem da tabela Arbitro
id_arbitro_assistente1	INTEGER	Obrigatório	-	Sim, vem da tabela Arbitro
id_arbitro_assistente2	INTEGER	Obrigatório	-	Sim, vem da tabela Arbitro
id_quarto_arbitro	INTEGER	Obrigatório	-	Sim, vem da tabela Arbitro

Tabela 13 - Partida

Atributo	Tipo de Dados	Restrição de Domínio	Restrição de Entidade	Restrição Referencial
id_partida	INTEGER	Obrigatório	Chave Primária	-
id_torneio	INTEGER	Obrigatório	-	Sim, da tabela Torneio
id_estadio	INTEGER	Obrigatório	-	Sim, da tabela Estádio
id_time_casa	INTEGER	Obrigatório	-	Sim, da tabela Time

id_time_fora	INTEGER	Obrigatório	-	Sim, da tabela Time
id_arbitragem	INTEGER	Obrigatório	-	Sim, da tabela Arbitragem
dataPartida	TIMESTAMP	Obrigatório	-	-
rodada	INTEGER	Obrigatório	-	-
fase	VARCHAR(50)	Obrigatório, valores: ('Primeiro Turno', 'Segundo Turno', 'Oitavas', 'Quartas', 'Semi', 'Final')	-	-
golsCasa	INTEGER	Obrigatório, >=0, valor padrão é 0	-	-
golsFora	INTEGER	Obrigatório, >=0, valor padrão é 0	-	-
status	VARCHAR(50)	Obrigatório, valores: ('Agendada','Em andamento','Encerrada','Cancelada'), valor padrão é "Agendada"	-	-
publico	INTEGER	Obrigatório, >=0, valor padrão é 0	-	-

Tabela 14 - EventoPartida

Atributo	Tipo de Dados	Restrição de Domínio	Restrição de Entidade	Restrição Referencial
id_partida	INTEGER	Obrigatório	Chave Primária	Sim, da tabela Partida
numero_evento	INTEGER	Obrigatório	Chave Primária	-
id_jogador	INTEGER	Obrigatório	-	Sim, da tabela Jogador
id_time	INTEGER	Obrigatório	-	Sim, da tabela Partida
tipo_evento	VARCHAR(30)	Obrigatório, valores: ('GOL','GOL_CONTRA','PENALTI_MARCADO','PENALTI_PERDIDO','CARTAO_AMARELO','CARTAO_VERMELHO','SUBSTITUICAO_ENTRADA','SUBSTITUICAO_SAIDA')	-	-
minuto	INTEGER	Obrigatório, $0 \leq \text{minuto} \leq 120$	-	-

Tabela 15 - Classificação

Atributo	Tipo de Dados	Restrição de Domínio	Restrição de Entidade	Restrição Referencial
id_classificacao	INTEGER	Obrigatório	Chave Primária	-
id_torneio	INTEGER	Obrigatório	-	Sim, da tabela Torneio
id_time	INTEGER	Obrigatório	-	Sim, da tabela Time
jogos	INTEGER	Obrigatório, ≥ 0 , valor padrão é 0	-	-
vitorias	INTEGER	Obrigatório, ≥ 0 , valor padrão é 0	-	-
empates	INTEGER	Obrigatório, ≥ 0 , valor padrão é 0	-	-
derrotas	INTEGER	Obrigatório, ≥ 0 , valor padrão é 0	-	-
gols_pro	INTEGER	Obrigatório, ≥ 0 , valor padrão é 0	-	-
gols_contra	INTEGER	Obrigatório, ≥ 0 , valor padrão é 0	-	-

saldo_gols	INTEGER	Obrigatório	-	-
pontos	INTEGER	Obrigatório, >=0, valor padrão é 0	-	-

Tabela 16 - Escalacao_time

Atributo	Tipo de Dados	Restrição de Domínio	Restrição de Entidade	Restrição Referencial
id_partida	INTEGER	Obrigatório	Chave Primária	Sim, da tabela Partida
id_jogador	INTEGER	Obrigatório	Chave Primaria	Sim, da tabela Jogador
id_time	INTEGER	Obrigatório	-	Sim, da tabela Time
titular	BOOLEAN	Obrigatório	-	-

4. Script Criação do Banco de Dados

4.1. Script Criação Tabelas

Código-Fonte 1 - Pessoa

```
SQL
-- Pessoa
CREATE TABLE Pessoa (
    id_pessoa INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    primeiroNome VARCHAR(50) NOT NULL,
    nomeMeio VARCHAR(50),
    ultimoNome VARCHAR(50) NOT NULL,
    dataNasc DATE NOT NULL,
    documentoIdentidade VARCHAR(20) UNIQUE,
    nacionalidade VARCHAR(50) NOT NULL,
    sexo CHAR(1) NOT NULL CHECK (sexo IN ('M', 'F'))
);
```

Código-Fonte 2 - Tabela Telefone_Pessoa

SQL

```
-- Telefone_Pessoa
CREATE TABLE Telefone_Pessoa (
    id_pessoa INTEGER NOT NULL,
    telefone VARCHAR(20) NOT NULL,
    PRIMARY KEY (id_pessoa, telefone),
    FOREIGN KEY (id_pessoa) REFERENCES Pessoa(id_pessoa)
);
```

Código-Fonte 3 - Tabela Email_Pessoa

SQL

```
-- Email_Pessoa
CREATE TABLE Email_Pessoa (
    id_pessoa INTEGER NOT NULL,
    email VARCHAR(60) NOT NULL,
    PRIMARY KEY (id_pessoa, email),
    FOREIGN KEY (id_pessoa) REFERENCES Pessoa(id_pessoa)
);
```

Código-Fonte 4 - Tabela Arbitro

SQL

```
-- Árbitro
CREATE TABLE Arbitro (
    id_pessoa INTEGER PRIMARY KEY,
    federacao VARCHAR(60) NOT NULL,
    tipoLicenca VARCHAR(50) NOT NULL,
    status VARCHAR(50) CHECK (status IN ('Ativo', 'Suspendido', 'Aposentado'))
    DEFAULT 'Ativo',
    FOREIGN KEY (id_pessoa) REFERENCES Pessoa(id_pessoa)
);
```

Código-Fonte 5 - Tabela Jogador

SQL

```
-- Jogador
CREATE TABLE Jogador (
    id_pessoa INTEGER PRIMARY KEY,
    posicao VARCHAR(50) NOT NULL CHECK (posicao IN (
        'Goleiro',
```

```

        'Zagueiro',
        'Lateral Direito',
        'Lateral Esquerdo',
        'Volante',
        'Meia Atacante',
        'Meia Central',
        'Ponta Direita',
        'Ponta Esquerda',
        'Centroavante',
        'Segundo Atacante'
    )),
    altura DECIMAL(4,2) NOT NULL,
    peso DECIMAL(5,2) NOT NULL,
    perna_dominante VARCHAR(50) NOT NULL CHECK (perna_dominante IN
('Direita', 'Esquerda', 'Ambas')),
    FOREIGN KEY (id_pessoa) REFERENCES Pessoa(id_pessoa)
);

```

Código-Fonte 6 - Tabela Treinador

SQL

```

-- Treinador
CREATE TABLE Treinador (
    id_pessoa INTEGER PRIMARY KEY,
    tipoLicenca VARCHAR(50) NOT NULL,
    numeroLicenca INTEGER NOT NULL UNIQUE,
    FOREIGN KEY (id_pessoa) REFERENCES Pessoa(id_pessoa)
);

```

Código-Fonte 7 - Tabela Time

SQL

```

-- Time
-- OBS: Substituí SERIAL por GENERATED AS IDENTITY,
-- pois PostgreSQL 15+ removeu o suporte ao tipo SERIAL.
CREATE TABLE Time (
    id_time INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    nome VARCHAR(100) NOT NULL,
    sigla VARCHAR(3) NOT NULL,
    cidade VARCHAR(100) NOT NULL,
    tipo VARCHAR(50) NOT NULL CHECK (tipo IN ('Clube', 'Seleção Nacional')),
    categoria VARCHAR(50) NOT NULL CHECK (categoria IN ('Profissional',
'Amador', 'Base')),
    genero VARCHAR(50) NOT NULL CHECK (genero IN ('Masculino', 'Feminino')),

```

```

        faixa_etaria VARCHAR(50) CHECK (faixa_etaria IN ('Principal', 'Sub-20',
'Sub-17', 'Sub-15')),
        dataFundacao DATE NOT NULL
    );

```

Código-Fonte 8 - Tabela ComissaoTecnica

SQL

```

-- Comissão Técnica
CREATE TABLE ComissaoTecnica (
    id_time INTEGER NOT NULL,
    id_pessoa INTEGER NOT NULL,
    funcao VARCHAR(50) NOT NULL CHECK (funcao IN ('Treinador', 'Auxiliar
Técnico', 'Massagista', 'Preparador Físico')),
    dataInicio DATE NOT NULL,
    dataFim DATE,
    PRIMARY KEY (id_time, id_pessoa, funcao),
    FOREIGN KEY (id_time) REFERENCES Time(id_time),
    FOREIGN KEY (id_pessoa) REFERENCES Pessoa(id_pessoa)
);

```

Código-Fonte 9 - Tabela Elenco

SQL

```

-- Elenco
CREATE TABLE Elenco (
    id_elenco INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    id_time INTEGER NOT NULL,
    id_jogador INTEGER NOT NULL,
    numeroCamisa INTEGER NOT NULL,
    dataInicio DATE NOT NULL,
    dataFim DATE,
    UNIQUE (id_time, id_jogador),
    FOREIGN KEY (id_time) REFERENCES Time(id_time),
    FOREIGN KEY (id_jogador) REFERENCES Jogador(id_pessoa)
);

```

Código-Fonte 10 - Tabela Torneio

SQL

```

-- Torneio
CREATE TABLE Torneio (
    id_torneio INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,

```

```

    nome VARCHAR(100) NOT NULL UNIQUE,
    tipo VARCHAR(50) NOT NULL CHECK (tipo IN ('Pontos Corridos', 'Mata-Mata',
'Grupos', 'Amistoso')),
    ano INTEGER NOT NULL,
    pais VARCHAR(50) NOT NULL,
    genero VARCHAR(50) NOT NULL CHECK (genero IN ('Masculino', 'Feminino')),
    faixa_etaria VARCHAR(50) CHECK (faixa_etaria IN ('Principal', 'Sub-20',
'Sub-17', 'Sub-15')),
    status VARCHAR(50) NOT NULL CHECK (status IN ('Ativo', 'Encerrado'))
DEFAULT 'Ativo'
);

```

Código-Fonte 11 - Tabela Estadio

SQL

```

-- Estadio
CREATE TABLE Estadio (
    idEstadio INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    nome VARCHAR(100) NOT NULL,
    cidade VARCHAR(100) NOT NULL,
    capacidade INTEGER NOT NULL,
    dono VARCHAR(100) NOT NULL
);

```

Código-Fonte 12 - Tabela Arbitragem

SQL

```

-- Arbitragem
CREATE TABLE Arbitragem (
    id_arbitragem INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    id_arbitro_principal INTEGER NOT NULL,
    id_arbitro_assistente1 INTEGER NOT NULL,
    id_arbitro_assistente2 INTEGER NOT NULL,
    id_quarto_arbitro INTEGER NOT NULL,
    FOREIGN KEY (id_arbitro_principal) REFERENCES Arbitro(id_pessoa),
    FOREIGN KEY (id_arbitro_assistente1) REFERENCES Arbitro(id_pessoa),
    FOREIGN KEY (id_arbitro_assistente2) REFERENCES Arbitro(id_pessoa),
    FOREIGN KEY (id_quarto_arbitro) REFERENCES Arbitro(id_pessoa)
);

```


Código-Fonte 13 - Tabela Partida

SQL

```
-- Partida
CREATE TABLE Partida (
    id_partida INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    id_torneio INTEGER NOT NULL,
    id_estadio INTEGER NOT NULL,
    id_time_casa INTEGER NOT NULL,
    id_time_fora INTEGER NOT NULL,
    id_arbitragem INTEGER NOT NULL,
    dataPartida TIMESTAMP NOT NULL,
    fase VARCHAR(50) NOT NULL CHECK (fase IN ('Primeiro Turno', 'Segundo
Turno', 'Oitavas', 'Quartas', 'Semi', 'Final')),
    rodada INTEGER NOT NULL,
    golsCasa INTEGER NOT NULL DEFAULT 0,
    golsFora INTEGER NOT NULL DEFAULT 0,
    status VARCHAR(50) CHECK(status IN ('Agendada', 'Em andamento',
'Encerrada', 'Cancelada')) NOT NULL DEFAULT 'Agendada',
    publico INTEGER NOT NULL DEFAULT 0,
    FOREIGN KEY (id_torneio) REFERENCES Torneio(id_torneio),
    FOREIGN KEY (id_estadio) REFERENCES Estadio(idEstadio),
    FOREIGN KEY (id_time_casa) REFERENCES Time(id_time),
    FOREIGN KEY (id_time_fora) REFERENCES Time(id_time),
    FOREIGN KEY (id_arbitragem) REFERENCES Arbitragem(id_arbitragem)
);
```

Código-Fonte 14 - Tabela EventoPartida

SQL

```
-- EventoPartida
CREATE TABLE EventoPartida (
    id_partida INTEGER NOT NULL,
    numero_evento INTEGER NOT NULL,
    id_jogador INTEGER NOT NULL,
    id_time INTEGER NOT NULL,
    tipo_evento VARCHAR(30) NOT NULL CHECK (tipo_evento IN (
        'GOL', 'GOL_CONTRA', 'PENALTI_MARCADO', 'PENALTI_PERDIDO',
        'CARTAO_AMARELO', 'CARTAO_VERMELHO',
        'SUBSTITUICAO_ENTRADA', 'SUBSTITUICAO_SAIDA'
    )),
    minuto INTEGER NOT NULL CHECK (minuto >= 0 AND minuto <= 120),
    PRIMARY KEY (id_partida, numero_evento),
    FOREIGN KEY (id_partida) REFERENCES Partida(id_partida) ON DELETE
CASCADE,
    FOREIGN KEY (id_jogador) REFERENCES Jogador(id_pessoa),
    FOREIGN KEY (id_time) REFERENCES Time(id_time)
);
```

Código-Fonte 15 - Tabela Classificacao

SQL

```
-- Classificacao
CREATE TABLE Classificacao (
    id_classificacao INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    id_torneio INTEGER NOT NULL,
    id_time INTEGER NOT NULL,
    jogos INTEGER NOT NULL CHECK (jogos >= 0) DEFAULT 0,
    vitorias INTEGER NOT NULL CHECK (vitorias >= 0) DEFAULT 0,
    empates INTEGER NOT NULL CHECK (empates >= 0) DEFAULT 0,
    derrotas INTEGER NOT NULL CHECK (derrotas >= 0) DEFAULT 0,
    gols_pro INTEGER NOT NULL CHECK (gols_pro >= 0) DEFAULT 0,
    gols_contra INTEGER NOT NULL CHECK (gols_contra >= 0) DEFAULT 0,
    saldo_gols INTEGER NOT NULL DEFAULT 0,
    pontos INTEGER NOT NULL CHECK (pontos >= 0) DEFAULT 0,
    FOREIGN KEY (id_torneio) REFERENCES Torneio(id_torneio),
    FOREIGN KEY (id_time) REFERENCES Time(id_time)
);
```

Código-Fonte 16 - Tabela Escalacao_time

SQL

```
-- Escalacao_time
CREATE TABLE Escalacao_time (
    id_partida INTEGER NOT NULL,
    id_jogador INTEGER NOT NULL,
    titular BOOLEAN NOT NULL,
    PRIMARY KEY (id_partida, id_jogador),
    FOREIGN KEY (id_partida) REFERENCES Partida(id_partida) ON DELETE
    CASCADE,
    FOREIGN KEY (id_jogador) REFERENCES Jogador(id_pessoa)
);
```

5. Scripts para Consulta

5.1. Consulta 1 - Ver todas as partidas com detalhes completos.

Essa consulta foi implementada para apresentar todas as partidas com informações completas, incluindo times, placares, estádio, arbitragem, fase, status e dados do torneio.

SQL

```
-- 1) Ver todas as partidas com detalhes completos, incluindo nomes dos
times, placares,
-- data, fase, status, estádio, árbitros, rodada e nome do torneio.
SELECT
    p.id_partida,
    t.nome AS torneio,
    p.rodada,
    tc.nome AS time_casa,
    tf.nome AS time_fora,
    p.golsCasa,
    p.golsFora,
    p.dataPartida,
    p.fase,
    p.status,
    e.nome AS estadio,
    CONCAT(ap1.primeiroNome, ' ', ap1.nomeMeio, ' ', ap1.ultimoNome) AS
arbitro_principal,
    CONCAT(ap2.primeiroNome, ' ', ap2.nomeMeio, ' ', ap2.ultimoNome) AS
arbitro_assistente1,
    CONCAT(ap3.primeiroNome, ' ', ap3.nomeMeio, ' ', ap3.ultimoNome) AS
arbitro_assistente2,
    CONCAT(ap4.primeiroNome, ' ', ap4.nomeMeio, ' ', ap4.ultimoNome) AS
quarto_arbitro
FROM Partida p
    JOIN Torneio AS t ON p.id_torneio = t.id_torneio
    JOIN Time AS tc ON p.id_time_casa = tc.id_time
    JOIN Time AS tf ON p.id_time_fora = tf.id_time
    JOIN Estadio AS e ON p.id_estadio = e.idEstadio
    JOIN Arbitragem AS a ON p.id_arbitragem = a.id_arbitragem
    JOIN Arbitro AS ar1 ON a.id_arbitro_principal = ar1.id_pessoa
    JOIN Arbitro AS ar2 ON a.id_arbitro_assistente1 = ar2.id_pessoa
    JOIN Arbitro AS ar3 ON a.id_arbitro_assistente2 = ar3.id_pessoa
    JOIN Arbitro AS ar4 ON a.id_quarto_arbitro = ar4.id_pessoa
    JOIN Pessoa AS ap1 ON ar1.id_pessoa = ap1.id_pessoa
    JOIN Pessoa AS ap2 ON ar2.id_pessoa = ap2.id_pessoa
    JOIN Pessoa AS ap3 ON ar3.id_pessoa = ap3.id_pessoa
    JOIN Pessoa AS ap4 ON ar4.id_pessoa = ap4.id_pessoa
ORDER BY p.dataPartida;
```

5.2. Consulta 2 - Gols marcados como mandante no Brasileiro Feminino A1

Essa consulta foi implementada para sumarizar quantos gols cada time marcou como mandante em um torneio específico, facilitando análises ofensivas.

```

SQL
SELECT
    t.nome AS time,
    SUM(p.golsCasa) AS gols_marcados
FROM Partida AS p
    JOIN Time AS t ON p.id_time_casa = t.id_time
WHERE p.id_torneio =
    (SELECT
        id_torneio
    FROM Torneio
    WHERE nome = 'Brasileiro Feminino A1')
GROUP BY t.nome
ORDER BY gols_marcados DESC;

```

5.3. Consulta 3 - Jogos dos quatro grandes de SP já encerrados

Essa consulta foi implementada para listar partidas finalizadas envolvendo os principais clubes paulistas, destacando confrontos e seus resultados.

```

SQL
SELECT
    tor.nome AS campeonato,
    p.rodada,
    tc.nome AS time_casa,
    p.golsCasa,
    p.golsFora,
    tf.nome AS time_fora
FROM Partida p
    JOIN Time AS tc ON p.id_time_casa = tc.id_time
    JOIN Time AS tf ON p.id_time_fora = tf.id_time
    JOIN Torneio AS tor ON p.id_torneio = tor.id_torneio
WHERE (tc.nome IN ('Corinthians', 'Santos', 'Palmeiras', 'São Paulo FC')
    OR tf.nome IN ('Corinthians', 'Santos', 'Palmeiras', 'São Paulo FC'))
    AND p.status = 'Encerrada'
ORDER BY p.rodada;

```

5.4. Consulta 4 - Árbitros que mais apitaram partidas no Campeonato Brasileiro Série A Masculino 2025

Essa consulta foi implementada para mostrar os árbitros que mais apitaram partidas no campeonato brasileiro.

SQL

-- 4) Árbitros que mais apitaram partidas no Campeonato Brasileiro Série A Masculino 2025

```
SELECT
    CONCAT(p.primeiroNome, ' ', COALESCE(p.nomeMeio || ' ', '' ),
    p.ultimoNome) AS nome_arbitro,
    COUNT(*) AS total_partidas
FROM Partida AS pa
    JOIN Arbitragem AS a ON pa.id_arbitragem = a.id_arbitragem
    JOIN Arbitro AS ar ON a.id_arbitro_principal = ar.id_pessoa
    JOIN Pessoa AS p ON ar.id_pessoa = p.id_pessoa
WHERE pa.id_torneio = (
    SELECT id_torneio
    FROM Torneio
    WHERE nome = 'Campeonato Brasileiro Série A'
        AND genero = 'Masculino'
        AND ano = 2025
)
GROUP BY nome_arbitro
ORDER BY total_partidas DESC;=
```

5.5. Consulta 5 - Estádios que nunca sediaram uma partida

Essa consulta foi implementada para identificar estádios cadastrados que ainda não receberam nenhum jogo, útil para auditorias e planejamento.

SQL

```
SELECT
    e.nome AS estadio,
    e.idEstadio
FROM Estadio AS e
    LEFT JOIN Partida AS p ON p.id_estadio = e.idEstadio
WHERE p.id_estadio IS NULL;
```

5.6. Consulta 6 - Artilharia do Brasileirão Série A Masculino 2025

Artilharia do Brasileirão Série A Masculino 2025

```

SQL
SELECT
    CONCAT(p.primeiroNome, ' ', COALESCE(p.nomeMeio, ''), ' ', p.ultimoNome)
AS nome_jogador,
    COUNT(*) AS total_gols
FROM EventoPartida AS e
    JOIN Partida AS pa ON e.id_partida = pa.id_partida
    JOIN Jogador AS j ON e.id_jogador = j.id_pessoa
    JOIN Pessoa AS p ON j.id_pessoa = p.id_pessoa
WHERE
    pa.id_torneio = (
        SELECT id_torneio
        FROM Torneio
        WHERE nome = 'Campeonato Brasileiro Série A'
            AND genero = 'Masculino'
            AND ano = 2025
    )
    AND e.tipo_evento = 'GOL'
GROUP BY p.primeiroNome, p.nomeMeio, p.ultimoNome
ORDER BY total_gols DESC;

```

5.7. Consulta 7 - Estádios com maior público médio

Essa consulta foi implementada para calcular a média de público por estádio, permitindo identificar locais com maior adesão de torcedores.

```

SQL
SELECT
    e.nome AS estadio,
    COALESCE(AVG(p.publico), 0) AS publico_medio --Tratar Null
FROM Estadio AS e
    LEFT JOIN Partida AS p ON p.id_estadio = e.idEstadio
GROUP BY e.nome
ORDER BY publico_medio DESC

```

5.8. Consulta 8 - Jogadores com mais tempo de clube

Essa consulta foi implementada para mostrar os atletas com maior permanência ativa no clube, avaliando longevidade e vínculo contratual.

SQL

```
-- 8) Jogadores com mais tempo de clube (contrato ainda ativo)
SELECT
    CONCAT(p.primeiroNome, ' ', COALESCE(p.nomeMeio || ' ', ''), p.ultimoNome)
AS nome_jogador,
    t.nome AS nome_time,
    e.dataInicio,
    (CURRENT_DATE - e.dataInicio) AS dias_no_clube
FROM Elenco AS e
    JOIN Jogador AS j ON e.id_jogador = j.id_pessoa
    JOIN Pessoa AS p ON j.id_pessoa = p.id_pessoa
    JOIN Time AS t ON e.id_time = t.id_time
WHERE e.dataFim IS NULL
ORDER BY dias_no_clube DESC;
```

5.9. Consulta 9 - Classificação atual do Brasileiro Feminino A1 2025

Essa consulta foi implementada para exibir a tabela atualizada de classificação do torneio, ordenando os times conforme critérios esportivos.

SQL

```
-- 9) Classificação atual do Brasileiro Feminino A1 2025
SELECT
    t.nome AS time,
    c.jogos,
    c.vitorias,
    c.empates,
    c.derrotas,
    c.gols_pro,
    c.gols_contra,
    c.saldo_gols,
    c.pontos
FROM Classificacao AS c
    JOIN Time AS t ON c.id_time = t.id_time
WHERE c.id_torneio = (
    SELECT id_torneio
    FROM Torneio
```

```

WHERE nome = 'Brasileiro Feminino A1'
AND ano = 2025
)
ORDER BY c.pontos DESC, c.saldo_gols DESC, c.gols_pro DESC;

```

5.10. Consulta 10 - Times com menos cartões recebidos

Essa consulta foi implementada para identificar os times mais disciplinados no campeonato, contabilizando cartões recebidos por cada equipe.

```

SQL
-- 10) Times com menos cartões recebidos no Brasileiro Feminino A1 2025
SELECT
    t.nome AS time,
    COUNT(*) AS total_cartoes
FROM EventoPartida AS e
    JOIN Jogador AS j ON e.id_jogador = j.id_pessoa
    JOIN Partida AS p ON e.id_partida = p.id_partida
    JOIN Elenco AS el ON el.id_jogador = j.id_pessoa
                        AND p.dataPartida BETWEEN el.dataInicio
                        AND COALESCE(el.dataFim, p.dataPartida)
    JOIN Time AS t ON t.id_time = el.id_time
WHERE e.tipo_evento IN ('CARTAO_AMARELO', 'CARTAO_VERMELHO')
AND p.id_torneio = (
    SELECT id_torneio
    FROM Torneio
    WHERE nome = 'Brasileiro Feminino A1'
    AND ano = 2025
)
GROUP BY t.nome
ORDER BY total_cartoes;

```

6. Gatilhos

6.1. Gatilho 1 - AtualizarClassificacao

Essa trigger foi implementada para atualizar automaticamente a tabela de classificação sempre que uma partida for encerrada, ajustando jogos, gols, vitórias, empates, derrotas e pontos.

SQL

```
-- 1) Atualizar classificação após partida
CREATE OR REPLACE FUNCTION AtualizarClassificacao()
RETURNS TRIGGER
AS $$
DECLARE
    v_id_torneio INTEGER := NEW.id_torneio;
    v_id_casa INTEGER := NEW.id_time_casa;
    v_id_fora INTEGER := NEW.id_time_fora;
    v_gols_casa INTEGER := NEW.golsCasa;
    v_gols_fora INTEGER := NEW.golsFora;
BEGIN
    -- Só processa se o status for "Encerrada"
    IF NEW.status = 'Encerrada' THEN
        -- Atualiza a linha de classificação de cada time
        -- Casa
        UPDATE Classificacao
        SET jogos = jogos + 1,
            gols_pro = gols_pro + v_gols_casa,
            gols_contra = gols_contra + v_gols_fora,
            saldo_gols = (gols_pro + v_gols_casa) - (gols_contra +
v_gols_fora)
        WHERE id_torneio = v_id_torneio
            AND id_time = v_id_casa;

        -- Fora
        UPDATE Classificacao
        SET jogos = jogos + 1,
            gols_pro = gols_pro + v_gols_fora,
            gols_contra = gols_contra + v_gols_casa,
            saldo_gols = (gols_pro + v_gols_fora) - (gols_contra +
v_gols_casa)
        WHERE id_torneio = v_id_torneio
            AND id_time = v_id_fora;

        -- Determina resultado e atualiza pontuação
        IF v_gols_casa > v_gols_fora THEN
            -- time que jogou em casa venceu
            UPDATE Classificacao
            SET vitorias = vitorias + 1,
                pontos = pontos + 3
            WHERE id_torneio = v_id_torneio
                AND id_time = v_id_casa;

            -- time que jogou fora perdeu
            UPDATE Classificacao
            SET derrotas = derrotas + 1
            WHERE id_torneio = v_id_torneio
                AND id_time = v_id_fora;
```

```

ELSIF v_gols_casa < v_gols_fora THEN
    -- time que jogou fora venceu
    UPDATE Classificacao
    SET vitorias = vitorias + 1,
        pontos = pontos + 3
    WHERE id_torneio = v_id_torneio
        AND id_time = v_id_fora;

    -- time que jogou em casa perdeu
    UPDATE Classificacao
    SET derrotas = derrotas + 1
    WHERE id_torneio = v_id_torneio
        AND id_time = v_id_casa;

ELSE
    -- empate
    UPDATE Classificacao
    SET empates = empates + 1,
        pontos = pontos + 1
    WHERE id_torneio = v_id_torneio
        AND id_time IN (v_id_casa, v_id_fora);
END IF;
END IF;

RETURN NEW;
END;
$$ LANGUAGE plpgsql;

-- Chamando a trigger após atualização do status da partida
CREATE TRIGGER trg_atualizar_classificacao
AFTER UPDATE OF status ON Partida
FOR EACH ROW
WHEN (NEW.status = 'Encerrada')
EXECUTE FUNCTION AtualizarClassificacao();

```

6.2. Gatilho 2 - ImpedirDuploVinculo

Essa trigger foi implementada para impedir que um jogador fique vinculado simultaneamente a dois times, verificando vínculos ativos antes de permitir um novo registro.

```

SQL

-- 2) Impedir duplo vínculo de jogador em times diferentes
CREATE OR REPLACE FUNCTION ImpedirDuploVinculo()
RETURNS TRIGGER AS $$
DECLARE

```

```

        v_ativo INTEGER;
BEGIN
    SELECT COUNT(*) INTO v_ativo
    FROM Elenco
    WHERE id_jogador = NEW.id_jogador AND dataFim IS NULL;

    IF v_ativo > 0 THEN
        RAISE EXCEPTION 'Jogador % já possui vínculo ativo em outro time.',
NEW.id_jogador;
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

-- Criando a trigger para impedir duplo vínculo
CREATE TRIGGER trg_impedir_duplo_vinculo
BEFORE INSERT ON Elenco
FOR EACH ROW
EXECUTE FUNCTION ImpedirDuploVinculo();

```

6.3. Gatilho 1 - ValidarEventoPartida

Essa trigger foi implementada para impedir o registro de eventos em partidas ainda não iniciadas, garantindo que apenas partidas em andamento ou encerradas aceitem eventos.

```

SQL
-- 3) Validar evento de partida somente se a partida estiver em andamento ou encerrada
CREATE OR REPLACE FUNCTION ValidarEventoPartida()
RETURNS TRIGGER AS $$
DECLARE
    v_status VARCHAR(50);
BEGIN
    SELECT status INTO v_status FROM Partida WHERE id_partida =
NEW.id_partida;

    IF v_status = 'Agendada' THEN
        RAISE EXCEPTION 'Não é possível registrar eventos antes da partida
começar (status atual: %)', v_status;
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

```

```

-- Criando a trigger para validar evento de partida
CREATE TRIGGER trg_validar_evento_partida
BEFORE INSERT ON EventoPartida
FOR EACH ROW
EXECUTE FUNCTION ValidarEventoPartida();

```

7. Procedimentos Armazenados (Stored Procedure)

7.1. Procedure 1 - CadastrarJogador

Essa procedure foi implementada para cadastrar um novo jogador garantindo que não exista outro com o mesmo documento, criando também registros opcionais de telefone e e-mail.

SQL

```

CREATE OR REPLACE PROCEDURE CadastrarJogador(
    p_primeiroNome VARCHAR,
    p_nomeMeio VARCHAR,
    p_ultimoNome VARCHAR,
    p_dataNasc DATE,
    p_nacionalidade VARCHAR,
    p_sexo CHAR,
    p_documentoIdentidade VARCHAR,
    p_posicao VARCHAR,
    p_altura DECIMAL(4,2),
    p_peso DECIMAL(5,2),
    p_perna_dominante VARCHAR,
    p_telefone VARCHAR DEFAULT NULL,
    p_email VARCHAR DEFAULT NULL
)
AS $$
DECLARE
    v_id_pessoa INTEGER;
BEGIN
    -- Verifica se a pessoa já existe pelo documento
    SELECT id_pessoa INTO v_id_pessoa
    FROM Pessoa
    WHERE documentoIdentidade = p_documentoIdentidade
    LIMIT 1;

    -- Se já existe, apenas avisa e sai
    IF v_id_pessoa IS NOT NULL THEN
        RAISE NOTICE 'Já existe no banco de dados. Nenhum cadastro foi
realizado.';
    
```

```

        RETURN;
    END IF;

    -- Insere nova pessoa
    INSERT INTO Pessoa (primeiroNome, nomeMeio, ultimoNome, dataNasc,
nacionalidade, sexo, documentoIdentidade)
    VALUES (p_primeiroNome, p_nomeMeio, p_ultimoNome, p_dataNasc,
p_nacionalidade, p_sexo, p_documentoIdentidade);

    -- Recupera o ID da pessoa recém-cadastrada
    SELECT id_pessoa INTO v_id_pessoa
    FROM Pessoa
    WHERE documentoIdentidade = p_documentoIdentidade
    LIMIT 1;

    -- Insere na tabela Jogador
    INSERT INTO Jogador (id_pessoa, posicao, altura, peso, perna_dominante)
    VALUES (v_id_pessoa, p_posicao, p_altura, p_peso, p_perna_dominante);

    -- (Opcional) Insere telefone
    IF p_telefone IS NOT NULL THEN
        INSERT INTO Telefone_Pessoa (id_pessoa, telefone)
        VALUES (v_id_pessoa, p_telefone);
    END IF;

    -- (Opcional) Insere e-mail
    IF p_email IS NOT NULL THEN
        INSERT INTO Email_Pessoa (id_pessoa, email)
        VALUES (v_id_pessoa, p_email);
    END IF;

    RAISE NOTICE 'Jogador % % (documento: %) cadastrado com ID %.',
        p_primeiroNome, p_ultimoNome, p_documentoIdentidade, v_id_pessoa;
END;
$$ LANGUAGE plpgsql;

-- Chamando a procedure para testar
CALL CadastrarJogador(
    'Gabriel', 'Fiorentino', 'Bressane',
    '1998-05-17',
    'Brasileiro',
    'M',
    '567.890.123-14',
    'Lateral Direito',
    1.75,
    75.5,
    'Direita',
    '11999999999',

```

```
'gabriel.bressane@email.com'  
);
```

7.2. Procedure 2 - TransferirJogador

Essa procedure foi implementada para transferir um jogador entre times, encerrando automaticamente o vínculo anterior e criando um novo registro de elenco para o time de destino.

```
SQL  
-- 2) Transferir jogador entre times  
CREATE OR REPLACE PROCEDURE TransferirJogador(  
    p_id_jogador INTEGER,  
    p_id_time_origem INTEGER,  
    p_id_time_destino INTEGER,  
    p_numeroCamisa INTEGER  
)  
AS $$  
DECLARE  
    v_vinculo_existente INTEGER;  
BEGIN  
    -- Verifica se o jogador tem um vínculo ativo no time de origem  
    SELECT COUNT(*) INTO v_vinculo_existente  
    FROM Elenco  
    WHERE id_time = p_id_time_origem  
        AND id_jogador = p_id_jogador  
        AND dataFim IS NULL;  
  
    IF v_vinculo_existente > 0 THEN  
        -- Encerra o vínculo anterior  
        UPDATE Elenco  
        SET dataFim = CURRENT_DATE  
        WHERE id_time = p_id_time_origem  
            AND id_jogador = p_id_jogador  
            AND dataFim IS NULL;  
  
        RAISE NOTICE 'Jogador % transferido do time % para o time %.',  
            p_id_jogador, p_id_time_origem, p_id_time_destino;  
    ELSE  
        RAISE NOTICE 'Jogador % não possuía vínculo anterior. Cadastrando  
primeiro time.', p_id_jogador;  
    END IF;  
  
    -- Cria novo vínculo  
    INSERT INTO Elenco (id_time, id_jogador, numeroCamisa, dataInicio)  
    VALUES (p_id_time_destino, p_id_jogador, p_numeroCamisa, CURRENT_DATE);
```

```

END;
$$ LANGUAGE plpgsql;

-- Exemplo de chamada da procedure
CALL TransferirJogador(120,null, 1, 23);

```

7.3. Procedure 3 - FinalizarPartida

Essa procedure foi implementada para registrar o placar final de uma partida e alterar seu status para “Encerrada”, disparando a trigger que atualiza a classificação.

```

SQL
-- 3) Finalizar partida (sempre que executado, atualiza a classificação via trigger)
CREATE OR REPLACE PROCEDURE FinalizarPartida(
    p_id_partida INT,
    p_gols_casa INT,
    p_gols_fora INT
)
AS $$
BEGIN
    UPDATE Partida
    SET golsCasa = p_gols_casa,
        golsFora = p_gols_fora,
        status = 'Encerrada'
    WHERE id_partida = p_id_partida;

    RAISE NOTICE ' Partida % finalizada: % x %', p_id_partida, p_gols_casa,
p_gols_fora;
END;
$$ LANGUAGE plpgsql;

CALL FinalizarPartida(25, 20, 1); -- Exemplo de chamada da procedure

```

8. Funções

8.1. Função 1 - Gols time torneio

Essa função foi implementada para retornar a quantidade total de gols marcados por um time em um torneio, consultando diretamente os valores consolidados na tabela de Classificação.

SQL

```
CREATE OR REPLACE PROCEDURE CadastrarJogador(
    p_primeiroNome VARCHAR,
    p_nomeMeio VARCHAR,
    p_ultimoNome VARCHAR,
    p_dataNasc DATE,
    p_nacionalidade VARCHAR,
    p_sexo CHAR,
    p_documentoIdentidade VARCHAR,
    p_posicao VARCHAR,
    p_altura DECIMAL(4,2),
    p_peso DECIMAL(5,2),
    p_perna_dominante VARCHAR,
    p_telefone VARCHAR DEFAULT NULL,
    p_email VARCHAR DEFAULT NULL
)
AS $$
DECLARE
    v_id_pessoa INTEGER;
BEGIN
    -- Verifica se a pessoa já existe pelo documento
    SELECT id_pessoa INTO v_id_pessoa
    FROM Pessoa
    WHERE documentoIdentidade = p_documentoIdentidade
    LIMIT 1;

    -- Se já existe, apenas avisa e sai
    IF v_id_pessoa IS NOT NULL THEN
        RAISE NOTICE 'Já existe no banco de dados. Nenhum cadastro foi realizado.';
        RETURN;
    END IF;

    -- Insere nova pessoa
    INSERT INTO Pessoa (primeiroNome, nomeMeio, ultimoNome, dataNasc,
nacionalidade, sexo, documentoIdentidade)
    VALUES (p_primeiroNome, p_nomeMeio, p_ultimoNome, p_dataNasc,
p_nacionalidade, p_sexo, p_documentoIdentidade);

    -- Recupera o ID da pessoa recém-cadastrada
    SELECT id_pessoa INTO v_id_pessoa
    FROM Pessoa
    WHERE documentoIdentidade = p_documentoIdentidade
    LIMIT 1;

    -- Insere na tabela Jogador
    INSERT INTO Jogador (id_pessoa, posicao, altura, peso, perna_dominante)
    VALUES (v_id_pessoa, p_posicao, p_altura, p_peso, p_perna_dominante);
```



```

-- (Opcional) Insere telefone
IF p_telefone IS NOT NULL THEN
    INSERT INTO Telefone_Pessoa (id_pessoa, telefone)
    VALUES (v_id_pessoa, p_telefone);
END IF;

-- (Opcional) Insere e-mail
IF p_email IS NOT NULL THEN
    INSERT INTO Email_Pessoa (id_pessoa, email)
    VALUES (v_id_pessoa, p_email);
END IF;

RAISE NOTICE 'Jogador % % (documento: %) cadastrado com ID %.',
    p_primeiroNome, p_ultimoNome, p_documentoIdentidade, v_id_pessoa;
END;
$$ LANGUAGE plpgsql;

-- Chamando a procedure para testar
CALL CadastrarJogador(
    'Gabriel', 'Fiorentino', 'Bressane',
    '1998-05-17',
    'Brasileiro',
    'M',
    '567.890.123-14',
    'Lateral Direito',
    1.75,
    75.5,
    'Direita',
    '11999999999',
    'gabriel.bressane@email.com'
);

```

8.2. Função 2 - Partidas vencidas pelo time em um torneio

Essa função foi implementada para calcular quantas partidas um time venceu em determinado torneio, considerando vitórias como mandante ou visitante.

```

SQL
-- 2) Transferir jogador entre times
CREATE OR REPLACE PROCEDURE TransferirJogador(
    p_id_jogador INTEGER,
    p_id_time_origem INTEGER,
    p_id_time_destino INTEGER,
    p_numeroCamisa INTEGER
)
AS $$
DECLARE

```

```

        v_vinculo_existente INTEGER;
BEGIN
    -- Verifica se o jogador tem um vínculo ativo no time de origem
    SELECT COUNT(*) INTO v_vinculo_existente
    FROM Elenco
    WHERE id_time = p_id_time_origem
        AND id_jogador = p_id_jogador
        AND dataFim IS NULL;

    IF v_vinculo_existente > 0 THEN
        -- Encerra o vínculo anterior
        UPDATE Elenco
        SET dataFim = CURRENT_DATE
        WHERE id_time = p_id_time_origem
            AND id_jogador = p_id_jogador
            AND dataFim IS NULL;

        RAISE NOTICE 'Jogador % transferido do time % para o time %.',
        p_id_jogador, p_id_time_origem, p_id_time_destino;
    ELSE
        RAISE NOTICE 'Jogador % não possuía vínculo anterior. Cadastrando
        primeiro time.', p_id_jogador;
    END IF;

    -- Cria novo vínculo
    INSERT INTO Elenco (id_time, id_jogador, numeroCamisa, dataInicio)
    VALUES (p_id_time_destino, p_id_jogador, p_numeroCamisa, CURRENT_DATE);

END;
$$ LANGUAGE plpgsql;

-- Exemplo de chamada da procedure
CALL TransferirJogador(120,null, 1, 23);

```

8.3. Função 3 - Top 10 jogadores com mais gol pelo clube

Essa função foi implementada para listar os 10 jogadores com mais gols por um clube, trazendo o nome do atleta e a quantidade total de gols registrados em eventos de partida.

```

SQL
-- 3) Finalizar partida (sempre que executado, atualiza a classificação via
trigger)
CREATE OR REPLACE PROCEDURE FinalizarPartida(
    p_id_partida INT,
    p_gols_casa INT,

```

```
        p_gols_fora INT
    )
AS $$
BEGIN
    UPDATE Partida
    SET golsCasa = p_gols_casa,
        golsFora = p_gols_fora,
        status = 'Encerrada'
    WHERE id_partida = p_id_partida;

    RAISE NOTICE ' Partida % finalizada: % x %', p_id_partida, p_gols_casa,
p_gols_fora;
END;
$$ LANGUAGE plpgsql;

CALL FinalizarPartida(25, 20, 1); -- Exemplo de chamada da procedure
```